Unit 4: Functions and Pointers

1. Modular Programming Using Functions

Definition

Modular programming is a programming approach in which a large program is divided into smaller parts called modules. In C language, these modules are generally created using functions. Each function performs a specific task, such as addition, subtraction, multiplication or displaying information. This makes the program easier to understand, write, test, debug and maintain.

Example

Suppose we want to create a calculator that performs addition, subtraction and multiplication.

Instead of writing all the statements inside the main() function, we can divide the program into three separate functions:

- add() – performs addition.
- subtract() – performs subtraction.
- multiply() – performs multiplication.

Each function performs its own task, and the main() function calls these functions whenever required.

Syntax

```
return_type function_name()
{
   // Statements
}
```

C Program

Program to perform arithmetic operations using separate functions.

```
#include <stdio.h>

void add()
{
   int a = 10, b = 5;
   printf("Addition = %d\n", a + b);
}

void subtract()
{
   int a = 10, b = 5;
   printf("Subtraction = %d\n", a - b);
}

void multiply()
{
   int a = 10, b = 5;
   printf("Multiplication = %d\n", a * b);
}

int main()
{
```

```
    add();
    subtract();
    multiply();

    return 0;
}
```

Output

```
Addition = 15
Subtraction = 5
Multiplication = 50
```

Explanation of the Program

1. `#include <stdio.h>` includes the standard input-output header file, which allows us to use the `printf()` function.
2. `void add()` defines a function named `add`. The function calculates the sum of two numbers and displays the result.
3. `void subtract()` defines another function that subtracts the second number from the first number.
4. `void multiply()` defines a function that multiplies two numbers and displays the result.
5. `int main()` is the main function from where program execution begins.
6. `add();` calls the addition function. When this statement executes, control moves to the `add()` function.
7. `subtract();` calls the subtraction function.
8. `multiply();` calls the multiplication function.
9. `return 0;` indicates that the program has completed successfully.

Advantages of Modular Programming

1. Code Reusability: A function can be called multiple times without writing the same code again.
2. Easy Debugging: Errors can be identified and corrected in individual functions.
3. Easy Maintenance: Changes can be made to one function without modifying the entire program.
4. Code Readability: Dividing a large program into smaller functions makes it easier to understand.
5. Reduces Code Repetition: Common operations can be written once and reused whenever required.

Important point: Modular programming divides a large program into smaller functions, where each function performs a specific task.

2. Function Declaration, Definition and Function Prototype

Definition

A function is a block of code that performs a particular task. In C programming, functions help divide a large program into smaller and manageable parts. Before using a function, the compiler should know its name, return type and parameters. This information is provided through a function declaration or prototype, while the actual statements of the function are written in its definition.

Three Main Parts of a Function

1. Function Declaration
2. Function Definition
3. Function Call

2.1 Function Declaration

A function declaration informs the compiler about the function name, return type and number and types of parameters. It tells the compiler that a particular function exists in the program. It does not contain the actual implementation or statements of the function. A function declaration is generally written before the main() function.

Syntax:

```
return_type function_name(parameter_list);
```

Example:

```
int add(int, int);
```

Explanation:

- int – indicates that the function returns an integer value.
- add – is the function name.
- int, int – indicates that the function accepts two integer arguments.
- ; – indicates the end of the declaration.

2.2 Function Definition

A function definition contains the actual code that performs the required operation. It specifies what the function should do when it is called. The function definition includes the function name, parameters, opening and closing braces, and statements. It may return a value to the calling function using the return statement.

Syntax:

```
return_type function_name(parameters)
{
   // Statements
   return value;
}
```

Example:

```
int add(int a, int b)
{
   int sum;
   sum = a + b;
   return sum;
}
```

Explanation:

- `int add(int a, int b)` defines a function named `add`.
- `a` and `b` are parameters that receive integer values.
- `int sum` declares a variable to store the result.
- `sum = a + b` performs addition.
- `return sum` sends the calculated result back to the calling function.

2.3 Function Call

A function call is a statement used to execute a function. When a function is called, program control moves to that function, executes its statements and returns to the calling location. A function can be called from the `main()` function or from another function. The values passed during a function call are known as arguments.

Syntax:

```
function_name(arguments);
```

Example:

```
result = add(10, 20);
```

Here, `10` and `20` are passed to the `add()` function, and the returned result is stored in `result`.

Complete C Program

Program to demonstrate function declaration, definition and function call.

```
#include <stdio.h>
 // Function Declaration
int add(int, int);

int main()
{
   int result;

   // Function Call
   result = add(10, 20);

   printf("Addition = %d", result);

   return 0;
}

// Function Definition
int add(int a, int b)
{
   return a + b;
}
```

Output

```
Addition = 30
```

Explanation of the Program

1. #include <stdio.h> includes the standard input-output header file.
2. int add(int, int); is the function declaration. It informs the compiler that a function named add accepts two integer values and returns an integer.
3. int main() is the starting point of program execution.
4. int result; declares an integer variable to store the returned value.
5. result = add(10, 20); calls the function by passing 10 and 20 as arguments.
6. int add(int a, int b) is the function definition. The parameters a and b receive the values 10 and 20.
7. return a + b; adds both values and returns 30 to the calling function.
8. printf() displays the result.
9. return 0; terminates the program successfully.

Difference Between Declaration, Definition and Function Call

Function Declaration	Function Definition	Function Call
Informs the compiler about a function.	Contains the actual implementation.	Executes the function.
Ends with a semicolon.	Contains a function body.	Used to invoke a function.
Does not contain statements.	Contains executable statements.	May pass arguments.
Example: int add(int, int);	Example: int add(int a,int b){return a+b;}	Example: add(10,20);

Important point: A function prototype is another name for a function declaration that specifies the function's interface, including its return type and parameter types. For example, int add(int, int);.

3. Function Categories in C

Definition

In C programming, functions are divided into different categories based on whether they accept arguments and whether they return a value. Arguments are the values passed to a function when it is called. A return value is the result sent back by the function after completing its task. Based on these two characteristics, functions are classified into four main categories.

Four Categories of Functions

1. Function with no arguments and no return value.
2. Function with arguments and no return value.
3. Function with no arguments and with return value.
4. Function with arguments and with return value.

3.1 Function with No Arguments and No Return Value

This type of function does not receive any values from the calling function. It also does not return any result to the calling function. The function performs its task using the statements and variables declared inside it. The return type of this function is generally void.

Syntax:

```
void function_name()
{
   // Statements
}
```

Example:

```
#include <stdio.h>

void display()
{
   printf("Welcome to C Programming");
}

int main()
{
   display();

   return 0;
}
```

Output:

```
Welcome to C Programming
```

Explanation:

- void display() defines a function that does not accept arguments and does not return a value.
- printf() displays the message inside the function.
- display(); calls the function from the main() function.
- After executing the function, control returns to main().

3.2 Function with Arguments and No Return Value

This type of function receives values from the calling function through parameters. It performs the required operation using the received values but does not send any result back. The return type is generally void. The result can be displayed directly inside the function using printf().

Syntax:

```
void function_name(data_type variable)
{
   // Statements
}
```

Example:

```
#include <stdio.h>

void add(int a, int b)
{
   printf("Addition = %d", a + b);
}

int main()
{
   add(10, 20);

   return 0;
}
```

Output:

```
Addition = 30
```

Explanation:

- `void add(int a, int b)` defines a function that accepts two integer arguments.
- `a` and `b` receive the values passed during the function call.
- `a + b` performs the addition operation.
- `printf()` displays the result directly.
- `add(10, 20);` calls the function by passing two values.
- Since the function has a `void` return type, it does not return a result to `main()`.

3.3 Function with No Arguments and With Return Value

This type of function does not receive any arguments from the calling function. However, it performs an operation and returns a result to the calling function. The returned value can be stored in a variable and used in other calculations. This type of function is useful when the function itself obtains or calculates the required value.

Syntax:

```
data_type function_name()
{
   // Statements
   return value;
}
```

Example:

```
#include <stdio.h>

int getNumber()
{
   int a = 50;
   return a;
}

int main()
{
   int result;

   result = getNumber();

   printf("Number = %d", result);

   return 0;
}
```

Output: Number = 50

Explanation:

- `int getNumber()` defines a function that does not accept arguments but returns an integer value.
- `int a = 50;` declares and initializes a variable.
- `return a;` sends the value 50 back to the calling function.
- `result = getNumber();` calls the function and stores the returned value in `result`.
- `printf()` displays the value stored in `result`.

3.4 Function with Arguments and With Return Value

This type of function receives values from the calling function and performs the required operation using those values. After completing the operation, it returns the result to the calling function. It is one of the commonly used function categories in C programming. The returned value can be stored in a variable or used directly in an expression.

Syntax:

```
data_type function_name(parameters)
{
   // Statements
   return value;
}
```

Example:

```
#include <stdio.h>

int add(int a, int b)
{
   return a + b;
}

int main()
{
   int result;

   result = add(10, 20);

   printf("Addition = %d", result);

   return 0;
}
```

Output:

```
Addition = 30
```

Explanation:

- `int add(int a, int b)` defines a function that accepts two integer arguments and returns an integer value.
- `add(10, 20)` passes 10 and 20 to the function.
- `a` receives 10 and `b` receives 20.
- `return a + b;` calculates the sum and returns 30.
- `result = add(10, 20);` stores the returned value in the variable `result`.
- `printf()` displays the final result.

Difference Between Four Function Categories

Category	Arguments	Return value
Category 1	No	No
Category 2	Yes	No
Category 3	No	Yes
Category 4	Yes	Yes

Important point: The four categories are classified based on the presence or absence of arguments and return values.

4. Parameter Passing and Return Values

Definition

Parameter passing is the process of sending values from one function to another function during a function call. The function that receives these values uses parameters to perform the required operation. A return value is the result sent back from a called function to the calling function. In C programming, parameters and return values help functions communicate with each other and make programs more organized and reusable.

4.1 Actual Parameters (Arguments)

Actual parameters are the values or variables supplied to a function when it is called. They are written inside the parentheses of the function call. These values are passed to the corresponding formal parameters of the function. Actual parameters are also commonly called arguments.

Example: `add(10, 20);`
Here:

- `10` is the first actual argument.
- `20` is the second actual argument.
- Both values are passed to the `add()` function.

4.2 Formal Parameters

Formal parameters are the variables declared in the function definition or function prototype. They receive the values passed by the actual arguments during a function call. These parameters are used inside the function to perform calculations or other operations. They act as local variables within the function.

Example:

```
int add(int a, int b)
{
   return a + b;
}
```

Here:

- `a` is the first formal parameter.
- `b` is the second formal parameter.
- They receive the values from the function call.

4.3 Return Values

A return value is the result sent from a function to the calling function after completing its operation. The return statement is used to send this value back. The data type of the returned value should match the function's declared return type, subject to C's normal type conversions. A function declared with void does not return a value.

Syntax:

```
return value;
```

Example:

```
int square(int n)
{
   return n * n;
}
```

If the function is called as:

```
result = square(5);
```

The returned value is 25, which is stored in result.

C Program

Program to demonstrate parameter passing and return values.

```
#include <stdio.h>

int multiply(int a, int b)
{
   int result;

   result = a * b;

   return result;
}

int main()
{
   int answer;

   answer = multiply(5, 4);

   printf("Multiplication = %d", answer);

   return 0;
}
```

Output:

```
Multiplication = 20
```

Explanation of the Program

1. `int multiply(int a, int b)` defines a function named `multiply` that accepts two integer parameters and returns an integer value.
2. `int result;` declares a local variable to store the multiplication result.
3. `result = a * b;` multiplies the values received through the parameters.
4. `return result;` sends the calculated result back to the calling function.
5. `int answer;` declares a variable in `main()` to store the returned value.
6. `answer = multiply(5, 4);` calls the function by passing 5 and 4 as actual arguments.
7. The function receives these values in `a` and `b`, calculates 20 and returns it.
8. `printf()` displays the value stored in `answer`.

Important Points

- Actual parameters are supplied during a function call.
- Formal parameters are declared in the function definition.
- The `return` statement sends a value back to the calling function.
- The function's return type specifies the type of value it is expected to return.

5. Scope, Lifetime and Storage Classes of Variables

Definition

In C programming, variables are used to store data values during program execution. Every variable has a scope, lifetime and storage class that determine where it can be accessed, how long it exists and how it is stored. Scope defines the portion of the program where a variable can be used. Lifetime specifies how long the variable remains in memory, while storage classes define important properties such as scope, lifetime and linkage.

5.1 Scope of Variables

Scope refers to the region of a program where a variable can be accessed or used. A variable declared inside a function or block is generally accessible only within that function or block. A variable declared outside all functions can be accessed by functions after its declaration, subject to its linkage and visibility. Scope helps control variable accessibility and avoids unwanted interference between different parts of a program.

Types of Scope

1. **Local Scope**
2. **Global Scope**
3. **Block Scope**
4. **Function Scope (mainly applicable to labels in C)**

A. Local Variable

A local variable is a variable declared inside a function. It can be accessed only within that function or its nested blocks. It is generally created when the function starts executing and ceases to exist when the function finishes. Local variables are useful for storing temporary data required by a particular function.

Example:

```
#include <stdio.h>

void display()
{
   int a = 10;

   printf("Value = %d", a);
}

int main()
{
   display();

   return 0;
}
```

Output:

```
Value = 10
```

Explanation:

- `int a = 10;` declares a local variable inside `display()`.
- The variable `a` can be accessed inside `display()`.
- It cannot be directly accessed inside `main()` because it is local to `display()`.

B. Global Variable

A global variable is a variable declared outside all functions in a C program. It can be accessed by multiple functions after its declaration, subject to its visibility and linkage. Its lifetime normally continues throughout the execution of the program. Global variables are useful when the same data needs to be shared among different functions.

Example:

```
#include <stdio.h>

int a = 100;

void display()
{
   printf("Value = %d\n", a);
}

int main()
{
   display();

   printf("Value = %d", a);

   return 0;
}
```

Output:

```
Value = 100
Value = 100
```

Explanation:

- int a = 100; declares a global variable outside all functions.
- display() accesses and displays the global variable.
- main() also accesses the same variable.
- Both functions can use the variable because it is declared globally.

5.2 Lifetime of Variables

Lifetime refers to the period during which a variable exists in memory while a program is running. Some variables are created when a function or block starts executing and destroyed when it finishes. Other variables remain available throughout the entire execution of the program. Understanding lifetime helps programmers know when a variable is created, when its value is retained and when its memory is no longer associated with it.

Types of Lifetime

- Automatic Lifetime: The variable exists while its block is executing. Ordinary local variables usually have automatic storage duration.
- Static Lifetime: The variable exists throughout the entire execution of the program. Global variables and static variables have static storage duration.
- Dynamic Lifetime: Memory is allocated during program execution using functions such as malloc() and released using free().

5.3 Storage Classes in C

Storage classes specify how variables and functions are stored and accessed in a program. They determine properties such as the scope, lifetime and linkage of identifiers. C commonly teaches four traditional storage classes: auto, register, static and extern. Each storage class has different characteristics and is used for different programming requirements.

A. Auto Storage Class

The auto storage class is the default storage class for ordinary local variables declared inside a block. These variables are created when the block starts executing and cease to exist when the block finishes. They can be accessed only within the block where they are declared. The keyword auto can be used explicitly, although it is rarely necessary.

Syntax: auto int a = 10;

Example:

```
#include <stdio.h>

int main()
{
   auto int a = 10;

   printf("Value = %d", a);

   return 0;
```

```
}
```

Output:

```
Value = 10
```

Explanation:

- auto int a = 10; declares an automatic local variable.
- The variable a is available inside the main() block.
- Its lifetime ends when the block finishes execution.

B. Register Storage Class

The register storage class is used to request that a variable be stored in a CPU register for faster access. It is traditionally used for variables that are accessed frequently, such as loop counters. However, the compiler may ignore this request and decide where to store the variable. The address-of operator & cannot be applied to an object declared with the register storage-class specifier.

Syntax:

```
register int i;
```

Example:

```
#include <stdio.h>

int main()
{
   register int i;

   for(i = 1; i <= 5; i++)
   {
      printf("%d\n", i);
   }

   return 0;
}
```

Output:

```
1
2
3
4
5
```

Explanation:

- register int i; declares a register variable.
- i is used as the loop counter.
- The for loop executes five times.
- Each iteration displays the current value of i.

C. Static Storage Class

The static storage class allows a local variable to retain its value between function calls. Unlike an ordinary local variable, a static local variable is initialized only once and exists throughout the execution of the program. Its scope remains limited to the block in which it is declared. It is useful when a function needs to remember a value from its previous execution.

Syntax:

```
static int a = 0;
```

Example:

```
#include <stdio.h>

void count()
{
   static int a = 0;

   a++;

   printf("%d\n", a);
}

int main()
{
   count();
   count();
   count();

   return 0;
}
```

Output:

```
1
2
3
```

Explanation:

1. static int a = 0; declares a static local variable and initializes it to zero.
2. a++; increases the value of a by one each time the function is called.
3. The first call changes a from 0 to 1.
4. The second call changes a from 1 to 2 because the previous value is retained.
5. The third call changes a from 2 to 3.
6. The output displays 1, 2 and 3.

Important: An ordinary local variable would be initialized again whenever the function is called if it is initialized inside the function. A static local variable retains its previous value.

D. Extern Storage Class

The extern storage class is used to declare a global variable or function that is defined elsewhere. It tells the compiler that the variable exists and has been defined in another place, possibly in another source file. It allows different parts of a program to access the same externally linked variable. An extern declaration usually does not allocate separate storage for the variable.

Syntax:

```
extern int a;
```

Example:

```
#include <stdio.h>

int a = 100;

int main()
{
   extern int a;

   printf("Value = %d", a);

   return 0;
}
```

Output:

```
Value = 100
```

Explanation:

- int a = 100; defines a global variable.
- extern int a; declares that the variable a is defined elsewhere.
- printf() accesses and displays the value of the variable.
- In this example, the definition appears earlier in the same source file. extern is also commonly used to access a global variable defined in another source file.

Difference Between Storage Classes

Storage class	Scope	Lifetime	Main purpose
auto	Block	Automatic	Ordinary local variables
register	Block	Automatic	Request fast access
static	Block or file, depending on declaration	Entire program	Retain values or restrict file visibility
extern	Depends on declaration and linkage	Usually entire program for the referenced global object	Refer to an externally defined variable

Important points:

- Local variables are generally accessible only within their block.
- Global variables are declared outside all functions.
- Static local variables retain their values between function calls.
- extern refers to a variable or function defined elsewhere.
- register is a request to the compiler, not a guarantee of CPU-register storage.

6. Recursion

Definition

Recursion is a programming technique in which a function calls itself to solve a problem. A recursive function repeatedly calls itself with a smaller or simpler input until a stopping condition is reached. This stopping condition is called the base condition or base case. Recursion is useful for solving problems that can be divided into smaller problems of the same type, such as finding factorials and calculating Fibonacci numbers.

How Recursion Works

A recursive function mainly contains two important parts:

1. Base Condition: The condition that stops the function from calling itself again. Without a proper base condition, recursion may continue indefinitely and cause a stack overflow.
2. Recursive Call: A statement in which the function calls itself with a modified or smaller input.

Example: Factorial of a Number

The factorial of a positive integer is the product of all positive integers from 1 up to that number.

For example:

[5! = 5 times 4 times 3 times 2 times 1 = 120]

The factorial can also be defined as:

[n! = n times (n-1)!]

Base condition:

[0! = 1]

Step-by-Step Working

To calculate factorial(5):

Function call	Calculation	Result
factorial(5)	5 × factorial(4)	Waits
factorial(4)	4 × factorial(3)	Waits
factorial(3)	3 × factorial(2)	Waits
factorial(2)	2 × factorial(1)	Waits
factorial(1)	1 × factorial(0)	Waits
factorial(0)	Base condition	1

Now the functions return their results in reverse order:

- factorial(1) = 1 × 1 = 1
- factorial(2) = 2 × 1 = 2
- factorial(3) = 3 × 2 = 6
- factorial(4) = 4 × 6 = 24
- factorial(5) = 5 × 24 = 120

Syntax

```
return_type function_name(parameters)
{
   if(condition)
   {
      return value;
   }
   else
   {
      return function_name(modified_parameter);
   }
}
```

C Program to find the factorial of a number using recursion.

```
#include <stdio.h>

int factorial(int n)
{
   if(n == 0)
   {
      return 1;
   }
   else
   {
      return n * factorial(n - 1);
   }
}

int main()
{
   int num, result;

   printf("Enter a number: ");
   scanf("%d", &num);

   result = factorial(num);

   printf("Factorial = %d", result);

   return 0;
}
```

Output : Enter a number: 5
Factorial = 120

Explanation of the Program

1. `#include <stdio.h>` includes the standard input-output header file.
2. `int factorial(int n)` defines a recursive function named `factorial` that accepts an integer and returns an integer.
3. `if(n == 0)` checks whether the input value has reached zero. This is the base condition.
4. `return 1;` returns 1 when `n` becomes zero because the factorial of zero is one.
5. `return n * factorial(n - 1);` calls the same function with a smaller value and multiplies the returned result by `n`.
6. `int num, result;` declares variables to store the input number and factorial result.
7. `scanf("%d", &num);` accepts a number from the user.
8. `result = factorial(num);` calls the recursive function and stores the returned factorial.
9. `printf()` displays the final factorial result.

Another Example: Sum of Natural Numbers Using Recursion

The sum of natural numbers is the addition of all positive integers starting from 1 up to a given number. Recursion can calculate this sum by adding the current number to the sum of all numbers before it. The function keeps calling itself with a smaller value until it reaches zero. This demonstrates how a problem can be divided into smaller versions of the same problem.

Program:

```
#include <stdio.h>

int sum(int n)
{
   if(n == 0)
   {
      return 0;
   }
   else
   {
      return n + sum(n - 1);
   }
}

int main()
{
   int num, result;

   printf("Enter a number: ");
   scanf("%d", &num);

   result = sum(num);

   printf("Sum = %d", result);

   return 0;
}
```

Output:

```
Enter a number: 5
Sum = 15
```

Working:

[sum(5) = 5 + 4 + 3 + 2 + 1 + 0 = 15]

Advantages of Recursion

1. Reduces Code Length: Some problems can be solved with fewer statements using recursive functions.
2. Easy Problem Solving: Complex problems can be divided into smaller problems of the same type.
3. Code Readability: Recursive solutions can be easier to understand for problems with naturally recursive structures.
4. Useful in Algorithms: Recursion is commonly used in tree traversal, divide-and-conquer algorithms and mathematical calculations.

Disadvantages of Recursion

1. Recursive function calls require additional memory on the function call stack.
2. Too many recursive calls can cause stack overflow.
3. Recursive solutions may be slower than iterative solutions because of repeated function calls.
4. A missing or incorrect base condition can result in infinite recursion.

Difference Between Recursion and Iteration

Recursion	**Iteration**
A function calls itself.	Repetition is performed using loops.
Uses a base condition to stop.	Uses a loop condition to stop.
Requires function call stack memory.	Usually uses less additional memory.
Can make recursive problems easier to express.	Often more efficient for simple repetitions.
Example: Factorial using a function calling itself.	Example: Factorial using a for loop.

Important point: Recursion is a technique in which a function calls itself until a base condition is satisfied.

7. Pointers in C

Definition

A pointer is a special type of variable that stores the memory address of another variable. Every variable is stored at a particular location in the computer's memory, and that location has an address. A pointer stores this address instead of storing an ordinary data value. Pointers are useful for accessing and modifying data through memory addresses, working with arrays, passing arguments to functions and managing dynamic memory.

Example

Consider the following declaration:

```
int a = 10;
int *p;
p = &a;
```

Here:

- a is an ordinary integer variable containing the value 10.
- &a gives the memory address of variable a.
- p is a pointer variable that stores the address of a.
- *p accesses the value stored at that address, which is 10.

Important Pointer Operators

C provides two important operators for working with pointers.

1. Address-of Operator (&)

The address-of operator is used to obtain the memory address of a variable. It is written before the variable name. The address can be stored in a pointer variable of a compatible type. For example, &a gives the address of the integer variable a.

2. Dereference Operator (*)

The dereference operator is used to access the value stored at the memory address held by a pointer. It is written before the pointer variable when accessing the pointed-to object. For example, if p stores the address of a, then *p gives the value of a.

Syntax

Pointer declaration:

```
data_type *pointer_name;
```

Example:

```
int *p;
float *x;
char *ch;
```

Assigning an address:

```
p = &a;
```

Accessing the value:

```
printf("%d", *p);
```

C Program

Program to demonstrate pointers.

```
#include <stdio.h>

int main()
{
   int a = 10;
   int *p;

   p = &a;

   printf("Value of a = %d\n", a);
   printf("Address of a = %p\n", (void *)&a);
   printf("Value of p = %p\n", (void *)p);
   printf("Value using pointer = %d\n", *p);

   return 0;
}
```

Output

```
Value of a = 10
Address of a = 0x...
Value of p = 0x...
Value using pointer = 10
```

The actual hexadecimal address will vary from one execution to another.

Explanation of the Program

1. `#include <stdio.h>` includes the standard input-output header file.
2. `int a = 10;` declares an integer variable and stores the value 10.
3. `int *p;` declares a pointer variable that can store the address of an integer.
4. `p = &a;` assigns the memory address of variable `a` to pointer `p`.
5. `printf("%d", a);` displays the value stored in variable `a`.
6. `printf("%p", (void *)&a);` displays the memory address of variable `a`.
7. `printf("%p", (void *)p);` displays the address stored inside pointer `p`.
8. `printf("%d", *p);` uses the dereference operator to access and display the value stored at that address.
9. `return 0;` terminates the program successfully.

Modifying a Variable Using a Pointer

A pointer can also be used to modify the value of another variable by accessing its memory location.

Example:

```
#include <stdio.h>

int main()
{
   int a = 10;
   int *p;

   p = &a;

   *p = 50;

   printf("Value of a = %d", a);

   return 0;
}
```

Output:

```
Value of a = 50
```

Explanation:

- Initially, `a` contains 10.
- `p = &a;` stores the address of `a` in pointer `p`.
- `*p = 50;` changes the value stored at that address to 50.
- Since `p` points to `a`, the value of `a` becomes 50.

Important Points

- A pointer stores a memory address.
- & is used to obtain the address of a variable.
- * is used to access the value at the stored address.
- A pointer should be initialized with a valid address before dereferencing it.
- A pointer should have a compatible data type.

8. Pointer Arithmetic

Definition

Pointer arithmetic refers to performing arithmetic operations on pointer variables. It allows programmers to move from one memory location to another location containing elements of the same data type. Unlike ordinary arithmetic, pointer arithmetic depends on the size of the data type pointed to by the pointer. It is commonly used when accessing array elements through pointers.

Pointer Arithmetic Operations

The main pointer arithmetic operations are:

1. Increment (p++)
2. Decrement (p--)
3. Addition (p + n)
4. Subtraction (p - n)
5. Difference between two pointers (p2 - p1)

8.1 Incrementing a Pointer (p++)

Incrementing a pointer means moving it to the next element of the data type it points to. When a pointer is incremented, its address increases by the size of the pointed-to data type. For example, if an integer occupies 4 bytes, incrementing an integer pointer moves it forward by 4 bytes. This operation is useful for accessing consecutive elements of an array.

Syntax:

```
p++;
```

8.2 Decrementing a Pointer (p--)

Decrementing a pointer means moving it to the previous element of the data type it points to. When a pointer is decremented, its address moves backward by the size of the pointed-to data type. For example, an integer pointer moves backward by 4 bytes on a system where sizeof(int) is 4. This operation is useful when accessing array elements in reverse order.

Syntax:

```
p--;
```

8.3 Adding an Integer to a Pointer (p + n)

An integer can be added to a pointer to move it forward by a specified number of elements. The pointer does not simply increase by that integer number of bytes. Instead, the movement depends on the size of the data type pointed to. For example, p + 2 moves an integer pointer forward by two integer elements.

Syntax:

```
p + n;
```

8.4 Subtracting an Integer from a Pointer (p - n)

An integer can be subtracted from a pointer to move it backward by a specified number of elements. The movement depends on the size of the pointed-to data type. For example, `p - 2` moves an integer pointer backward by two integer elements. This operation is commonly used when moving backwards through an array.

Syntax:

```
p - n;
```

8.5 Difference Between Two Pointers

The difference between two pointers gives the number of elements between them when both pointers refer to elements of the same array, or one position past it. The result is measured in elements, not bytes. Pointer subtraction is valid only when both pointers refer to the same array object or its one-past position.

Syntax:

```
difference = p2 - p1;
```

Example of Pointer Arithmetic

Consider the following array:

```
int a[4] = {10, 20, 30, 40};
int *p = a;
```

Assume each integer occupies 4 bytes and the first element is stored at address 1000.

Array element	Value	Illustrative address
a[0]	10	1000
a[1]	20	1004
a[2]	30	1008
a[3]	40	1012

If `p` points to `a[0]`:

- `*p` gives 10.
- `*(p + 1)` gives 20.
- `*(p + 2)` gives 30.
- `*(p + 3)` gives 40.

C Program

Program to demonstrate pointer increment and decrement.

```
#include <stdio.h>

int main()
{
   int a[4] = {10, 20, 30, 40};
   int *p;

   p = a;

   printf("First element = %d\n", *p);

   p++;

   printf("Second element = %d\n", *p);

   p++;

   printf("Third element = %d\n", *p);

   p--;

   printf("After decrement = %d\n", *p);

   return 0;
}
```

Output

```
First element = 10
Second element = 20
Third element = 30
After decrement = 20
```

Explanation of the Program

1. `int a[4] = {10, 20, 30, 40};` declares an integer array containing four elements.
2. `int *p;` declares an integer pointer.
3. `p = a;` stores the address of the first array element in the pointer. In this expression, the array name is converted to a pointer to its first element.
4. `printf("%d", *p);` displays the first element because `p` points to `a[0]`.
5. `p++;` moves the pointer to the next integer element, `a[1]`.
6. `p++;` moves the pointer to `a[2]`.
7. `p--;` moves the pointer backward to `a[1]`.
8. `return 0;` terminates the program.

Important Points

- Pointer arithmetic depends on the size of the pointed-to data type.
- Increment moves a pointer to the next element.
- Decrement moves a pointer to the previous element.
- Pointer addition and subtraction must remain within the same array object or its one-past position.
- Adding two pointers or multiplying pointers is not permitted in C.

9. Relationship Between Arrays and Pointers

Definition

Arrays and pointers are closely related in C programming because an array name is converted to a pointer to its first element in most expressions. This allows array elements to be accessed using pointer notation. A pointer can store the address of an array element and move between elements using pointer arithmetic. Although arrays and pointers are closely connected, an array is a fixed collection of elements, whereas a pointer is a separate variable that stores an address.

Example

Consider the following declaration:

```
int a[3] = {10, 20, 30};
int *p;

p = a;
```

Here:

- `a` is an integer array containing three elements.
- `p` is an integer pointer.
- `p = a;` makes `p` point to the first element of the array.
- `*p` accesses the first element.
- `*(p + 1)` accesses the second element.
- `*(p + 2)` accesses the third element.

Accessing Array Elements Using Index and Pointer Notation

The same array element can be accessed using either array indexing or pointer notation.

Array notation	Pointer notation	Value
a[0]	*(a + 0)	10
a[1]	*(a + 1)	20
a[2]	*(a + 2)	30

The following expressions are equivalent:

```
a[i]
*(a + i)
```

C Program

Program to access array elements using pointers.

```
#include <stdio.h>

int main()
{
   int a[5] = {10, 20, 30, 40, 50};
   int *p;
   int i;

   p = a;

   printf("Array elements are:\n");

   for(i = 0; i < 5; i++)
   {
      printf("%d\n", *(p + i));
   }

   return 0;
}
```

Output

```
Array elements are:
10
20
30
40
50
```

Explanation of the Program

1. `int a[5] = {10, 20, 30, 40, 50};` declares an array containing five integer values.
2. `int *p;` declares a pointer that can store the address of an integer.
3. `int i;` declares a loop control variable.
4. `p = a;` assigns the address of the first array element to pointer `p`.
5. `for(i = 0; i < 5; i++)` repeats the statements five times to access all array elements.
6. `*(p + i)` calculates the address of the element at position `i` and accesses its value.
7. `printf()` displays each array element.
8. `return 0;` terminates the program successfully.

Difference Between Arrays and Pointers

Array	**Pointer**
An array stores a collection of elements of the same data type.	A pointer stores the address of another object.
Array size is fixed after declaration.	A pointer can be made to point to different compatible objects.
The array name is not a modifiable variable.	A pointer variable can be reassigned.
Array elements are accessed using indexes.	Elements can be accessed using pointer arithmetic.
`sizeof(array)` gives the total size of the array in bytes when used in its scope.	`sizeof(pointer)` gives the size of the pointer variable.

Important points:

- An array name is converted to a pointer to its first element in most expressions.
- a[i] and *(a + i) access the same element.
- An array and a pointer are not the same thing.
- When an array is passed to a function, it is adjusted to a pointer parameter.

10. Passing Arguments by Value and by Address

Definition

Argument passing is the process of supplying values to a function when it is called. In C programming, arguments can be passed by value or by passing the address of a variable through a pointer. In pass by value, the function receives a copy of the original value, so changes made inside the function do not affect the original variable. In the address-based approach, the function receives a pointer containing the variable's address and can modify the original value through that pointer.

There are two commonly studied methods:

1. Call by Value
2. Call by Address (using pointers)

10.1 Call by Value

Call by value is a method of passing arguments in which a copy of the actual argument is supplied to the function. The function works with its own copy of the value rather than directly accessing the original variable. Any changes made to the parameter inside the function will not affect the original variable in the calling function. This method is useful when the original data should remain unchanged.

Syntax:

```
function_name(variable);

void function_name(data_type parameter)
{
   // Statements
}
```

Example

Suppose the variable a contains 10. When we pass a to a function, the function receives a copy of 10. If the function changes its parameter to 50, the original variable a still contains 10.

C Program

Program to demonstrate call by value.

```
#include <stdio.h>

void change(int x)
{
   x = 50;

   printf("Value inside function = %d\n", x);
}

int main()
{
   int a = 10;

   printf("Before function call = %d\n", a);

   change(a);

   printf("After function call = %d\n", a);

   return 0;
}
```

Output

```
Before function call = 10
Value inside function = 50
After function call = 10
```

Explanation of the Program

1. `void change(int x)` defines a function named `change` that accepts an integer parameter.
2. `int a = 10;` declares a variable in the `main()` function and initializes it with 10.
3. `change(a);` calls the function and passes the value of `a`.
4. The function receives a copy of the value 10 in parameter `x`.
5. `x = 50;` changes the copied value inside the function.
6. `printf()` inside `change()` displays 50.
7. After the function finishes, control returns to `main()`.
8. The original variable `a` still contains 10 because only its copy was modified.

10.2 Call by Address

Call by address is a method in which the memory address of a variable is passed to a function using a pointer. The function receives this address through a pointer parameter and can access the original variable using the dereference operator. Changes made through the pointer directly affect the original variable. This method is useful when a function needs to modify the values of variables declared in another function.

Syntax:

```
function_name(&variable);

void function_name(data_type *pointer)
{
   // Statements
}
```

Example

Suppose the variable a contains 10. We pass its address to a function using &a. The function receives the address through a pointer and changes the value to 50 using *x = 50. Since the pointer accesses the original variable, its value becomes 50.

C Program

Program to demonstrate call by address.

```
#include <stdio.h>

void change(int *x)
{
   *x = 50;

   printf("Value inside function = %d\n", *x);
}

int main()
{
   int a = 10;

   printf("Before function call = %d\n", a);

   change(&a);

   printf("After function call = %d\n", a);

   return 0;
}
```

Output

```
Before function call = 10
Value inside function = 50
After function call = 50
```

Explanation of the Program

1. void change(int *x) defines a function that accepts an integer pointer as its parameter.
2. int a = 10; declares a variable and initializes it with 10.
3. change(&a); calls the function and passes the memory address of variable a.
4. The pointer x receives the address of a.
5. *x = 50; accesses the original variable through the pointer and changes its value to 50.
6. printf() inside the function displays the modified value.
7. After the function finishes, control returns to main().
8. printf() displays 50 because the original variable a has been modified.

Difference Between Call by Value and Call by Address

Call by Value	Call by Address
A copy of the value is passed.	The address of the variable is passed.
Uses ordinary parameters.	Uses pointer parameters.
Changes do not affect the original variable.	Changes through the pointer affect the original variable.
Does not require pointers.	Requires pointers to access the original variable.
Example: `change(a);`	Example: `change(&a);`

Important concept: Technically, C uses pass by value in both cases. In call by address, the value being passed is the memory address, which the function receives in a pointer parameter. C does not have true pass-by-reference parameters.

11. Applications of Pointers in C

Definition

Pointers are important features of C programming because they allow programmers to access and manipulate data through memory addresses. They provide an efficient way to work with arrays, strings, functions and dynamically allocated memory. Pointers also allow functions to modify original variables by receiving their addresses. They are widely used in data structures and system-level programming because they provide direct control over memory.

Major Applications of Pointers

1. Accessing Variables: Pointers can access the values of variables through their memory addresses.
2. Modifying Variables: Pointers allow functions to modify original variables by accessing their addresses.
3. Working with Arrays: Pointers can access array elements using pointer arithmetic.
4. String Handling: Pointers can be used to access and manipulate character arrays and strings.
5. Dynamic Memory Allocation: Pointers store addresses returned by memory allocation functions such as `malloc()`, `calloc()` and `realloc()`.
6. Passing Arguments to Functions: Pointers allow functions to access and modify the original variables of the calling function.
7. Implementing Data Structures: Pointers are used to create linked lists, trees, graphs, stacks and queues.
8. Working with Structures: Pointers can store the address of structure variables and access their members.

11.1 Modifying Variables Using Pointers

Pointers allow a function to modify the original value of a variable by accessing its memory address. Instead of passing the complete variable value, the address is passed to the function. The pointer parameter receives that address and uses the dereference operator to change the original value. This is useful when the function needs to update variables declared outside its own block.

Example:

```
#include <stdio.h>

int main()
{
   int a = 10;
   int *p;

   p = &a;

   *p = 100;

   printf("Modified value = %d", a);

   return 0;
}
```

Output:

```
Modified value = 100
```

Explanation:

- `int a = 10;` declares an integer variable.
- `int *p;` declares an integer pointer.
- `p = &a;` stores the address of `a` in the pointer.
- `*p = 100;` changes the original value of `a` to 100.
- `printf()` displays the modified value.

11.2 Swapping Two Numbers Using Pointers

Swapping means exchanging the values of two variables. Pointers make it possible to exchange the original values by passing their addresses to a function. The function receives two pointer parameters and uses a temporary variable to exchange the values stored at those addresses. After the function executes, the original variables contain the exchanged values.

Example:

Before swapping:

- a = 10
- b = 20

After swapping:

- a = 20
- b = 10

C Program:

```
#include <stdio.h>

void swap(int *x, int *y)
{
   int temp;

   temp = *x;
   *x = *y;
   *y = temp;
}

int main()
{
   int a = 10, b = 20;

   printf("Before swapping:\n");
   printf("a = %d\n", a);
   printf("b = %d\n", b);

   swap(&a, &b);

   printf("After swapping:\n");
   printf("a = %d\n", a);
   printf("b = %d\n", b);

   return 0;
}
```

Output:

```
Before swapping:
a = 10
b = 20
After swapping:
a = 20
b = 10
```

Explanation of the Program

1. `void swap(int *x, int *y)` defines a function that accepts two integer pointers.
2. `int temp;` declares a temporary variable used to store one value during swapping.
3. `temp = *x;` stores the value of the first variable in `temp`.
4. `*x = *y;` copies the second variable's value into the first variable.
5. `*y = temp;` copies the original first value into the second variable.
6. `int a = 10, b = 20;` declares and initializes two integer variables.
7. `swap(&a, &b);` passes the addresses of both variables to the function.
8. The function exchanges the original values through the pointers.
9. The `printf()` statements display the values before and after swapping.

11.3 Dynamic Memory Allocation

Dynamic memory allocation is the process of allocating memory during program execution according to the program's requirements. Pointers are used to store the addresses of the allocated memory blocks. C provides memory management functions such as malloc(), calloc(), realloc() and free() through the stdlib.h header file. Dynamic memory allocation is useful when the required amount of memory is not known at compile time.

Important Dynamic Memory Functions

Function	Purpose
malloc()	Allocates a block of memory without initializing its contents.
calloc()	Allocates memory for multiple elements and initializes all bytes to zero.
realloc()	Changes the size of a previously allocated memory block.
free()	Releases dynamically allocated memory.

Syntax:

```
pointer = (data_type *)malloc(size);
```

In modern C, the cast is not required:

```
pointer = malloc(size);
```

C Program : Program to allocate memory dynamically using malloc().

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int *p;
   int i;

   p = malloc(3 * sizeof(int));

   if(p == NULL)
   {
      printf("Memory allocation failed");
      return 1;
   }

   for(i = 0; i < 3; i++)
   {
      printf("Enter a number: ");
      scanf("%d", &p[i]);
   }

   printf("Entered numbers are:\n");

   for(i = 0; i < 3; i++)
   {
      printf("%d\n", p[i]);
   }

   free(p);
```

```
    return 0;
}
```

Output:

```
Enter a number: 10
Enter a number: 20
Enter a number: 30
Entered numbers are:
10
20
30
```

Explanation of the Program

1. #include <stdio.h> includes input-output functions.
2. #include <stdlib.h> includes memory allocation functions such as malloc() and free().
3. int *p; declares an integer pointer.
4. p = malloc(3 * sizeof(int)); requests memory sufficient to store three integer values.
5. if(p == NULL) checks whether memory allocation failed.
6. The first for loop accepts three integer values from the user and stores them in dynamically allocated memory.
7. p[i] accesses each allocated integer element.
8. The second for loop displays all three entered values.
9. free(p); releases the allocated memory after its use.
10. return 0; terminates the program successfully.

11.4 Working with Arrays Using Pointers

Pointers can access array elements without using the array index directly. The address of the first element can be stored in a pointer, and pointer arithmetic can be used to move between elements. This makes it possible to read, display and modify array elements using pointers. It is particularly useful when arrays are passed to functions.

Example:

```
#include <stdio.h>

int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a;
    int i;

    for(i = 0; i < 5; i++)
    {
        printf("%d\n", *(p + i));
    }

    return 0;
}
```

Output:

```
10
20
30
40
50
```

Explanation:

- `int a[5]` declares an array of five integers.
- `int *p = a;` makes the pointer point to the first element.
- The `for` loop accesses each element.
- `*(p + i)` retrieves the value at the current array position.
- `printf()` displays the values one by one.

Summary of Applications of Pointers

Application	Use of pointers
Variable modification	Changes values through memory addresses.
Swapping	Exchanges original variable values.
Arrays	Accesses elements using pointer arithmetic.
Strings	Accesses and manipulates characters.
Dynamic memory	Manages memory allocated during execution.
Functions	Passes addresses to functions.
Data structures	Creates linked lists, trees and graphs.
Structures	Accesses structure members through pointers.